

Worksheet No. - 02

Student Name: Sukhleen Kaur

UID: 25MCA20031

Branch: MCA

Section/Group: 25MCA-6B

Semester: 2nd

Date of Performance: 27.01.2026

Subject Name: Design and Analysis of Algorithm Lab **Subject Code:** 25CAP-612

Aim/Overview of the practical: To implement a parallel merge sort algorithm and evaluate its performance by measuring the sorting time for different input sizes.

Objective: Using Open, Implement a parallelized merge sort algorithm to sort a given set of elements and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted and plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

Procedure/Algorithm/Code

Algorithm:

1. Start the program.
2. Divide the array into two halves recursively until each subarray has one element.
3. Sort the left and right subarrays using recursive calls.
4. Merge the sorted subarrays by comparing elements and arranging them in order.
5. Repeat until the entire array is sorted.
6. Display the sorted array.
7. Stop.

Code:

```
#include <iostream>

using namespace std;

void merge(int arr[], int p, int q, int r) {

    int n1 = q - p + 1;

    int n2 = r - q;
```

```
int L[n1], M[n2];

int i, j, k;

for (i = 0; i < n1; i++)

    L[i] = arr[p + i];

for (j = 0; j < n2; j++)

    M[j] = arr[q + 1 + j];

i = 0;

j = 0;

k = p;

while (i < n1 && j < n2) {

    if (L[i] <= M[j]) {

        arr[k] = L[i];

        i++;

    } else {

        arr[k] = M[j];

        j++;

    }

    k++;

}

while (i < n1) {

    arr[k] = L[i];

    i++;

    k++;

}
```

```
while (j < n2) {  
    arr[k] = M[j];  
    j++;  
    k++;  
}  
}  
  
void mergeSort(int arr[], int l, int r) {  
    if (l < r) {  
        int m = l + (r - l) / 2;  
        mergeSort(arr, l, m);  
        mergeSort(arr, m + 1, r);  
        merge(arr, l, m, r);  
    }  
}  
  
void printArray(int arr[], int size) {  
    for (int i = 0; i < size; i++)  
        cout << arr[i] << " ";  
    cout << endl;  
}  
  
int main() {  
    int arr[] = {6, 5, 12, 10, 9, 1};  
    int size = sizeof(arr) / sizeof(arr[0]);  
    mergeSort(arr, 0, size - 1);  
}
```

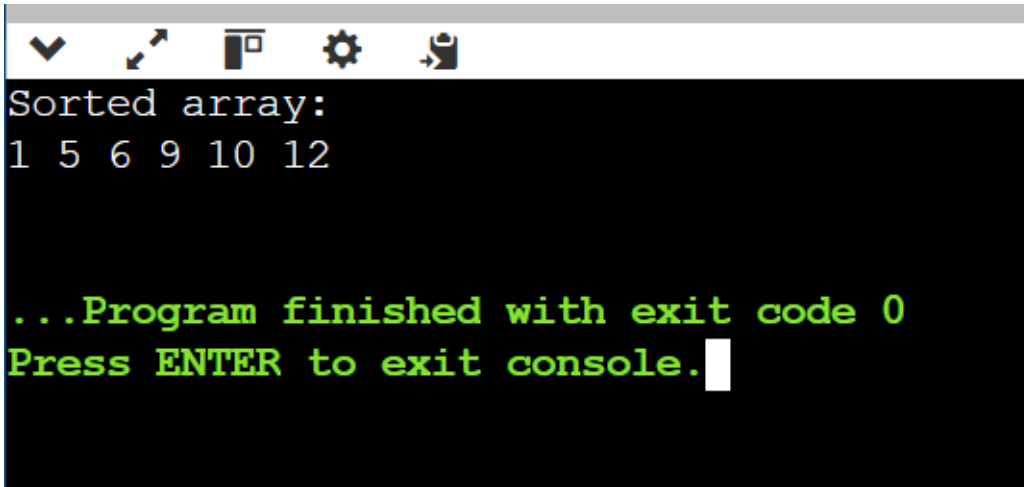
```
cout << "Sorted array:\n";
```

```
printArray(arr, size);
```

```
return 0;
```

```
}
```

Output

A screenshot of a Windows-style console window. The title bar is light gray and contains several icons: a checkmark, a magnifying glass, a square, a gear, and a document. The console background is black with white text. The output shows 'Sorted array:' followed by a new line and the numbers '1 5 6 9 10 12'. Below this, in green text, it says '...Program finished with exit code 0' and 'Press ENTER to exit console.' with a white cursor at the end of the second line.

```
Sorted array:  
1 5 6 9 10 12  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Learning outcomes (What I have learnt):

1. Understood the concept of Merge Sort and the divide and conquer approach.
2. Implemented the Merge Sort algorithm in C++ using recursion.
3. Analyzed the time complexity and performance of Merge Sort.